

[/](#) [Мое](#) / [Мои проекты](#) / [AI-агент службы поддержки](#)

Проект: AI-агент службы поддержки

15 августа
2026 г.

> НЕДОСТ...

НЕДОСТ...

В ПРОЦ...

[Репозиторий](#)[Помощь](#)

Подготовка



1



2



3



4



5



6



7



8



9



10



Проверка

Чему учимся

Собирать полноценное LLM-приложение на Yandex AI Studio и доводить его до работающего продукта в облаке:

- Поднимать агента на модели YandexGPT и подключать его к почтовому каналу (IMAP/SMTP);
- Подключать RAG — поиск ответа по корпоративной базе знаний;
- Писать собственный MCP-инструмент и хранить данные в YDB Serverless;
- Запускать фоновый workflow по расписанию;
- Защищаться от prompt injection и утечек персональных данных (PII);
- Налаживать наблюдаемость — трейсы вызовов и учёт токенов.

Описание проекта

Вы разрабатываете **Help Desk-агента** — почтового агента службы поддержки для сотрудников компании. Агент принимает обращение по email, ищет ответ в корпоративной базе знаний (документация, FAQ, регламенты) и, если готового ответа нет, создаёт тикет и сохраняет переписку в базе данных. Всё решение работает внутри контура Yandex Cloud — без внешних трекеров вроде Яндекс Трекера.

Как это выглядит для пользователя:

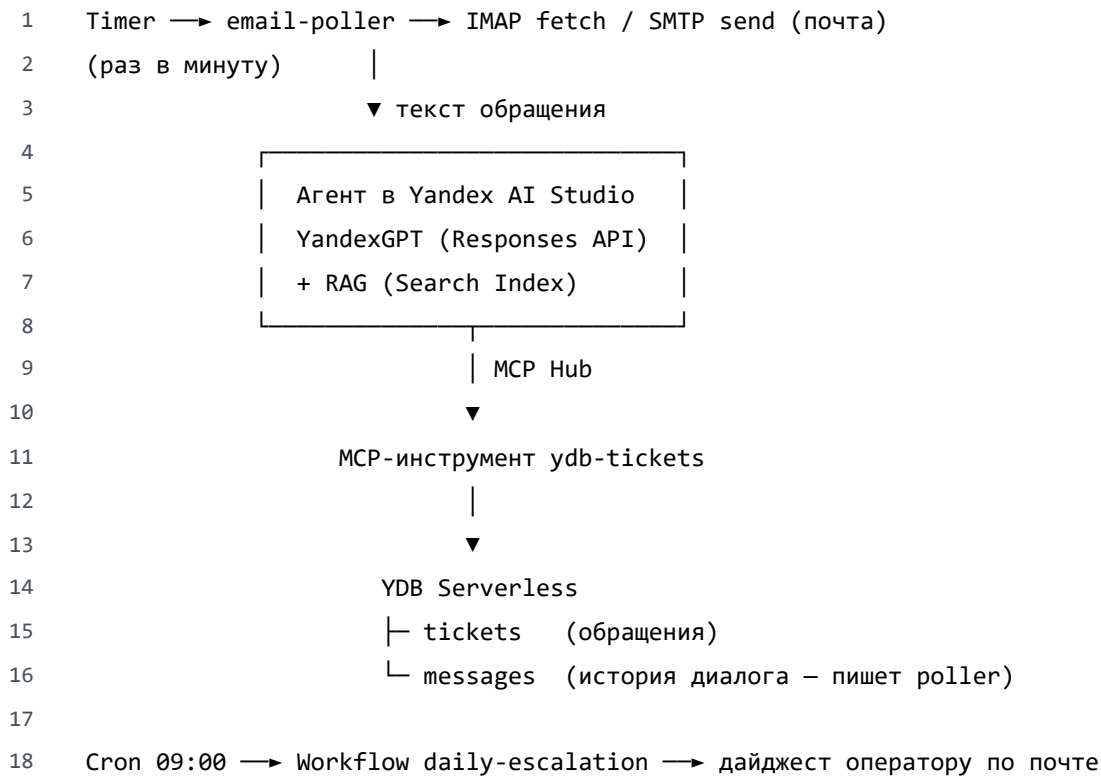


Все курсы

- «Сломался принтер, заведи заявку» → ответа в базе нет, бот создает тикет и возвращает его номер.
- Раз в день бот сам присылает оператору дайджест тикетов, которые долго остаются без ответа (авто-эскалация).

Это сквозной проект профессии LLM-инженер. Вы пройдёте путь от пустого облачного аккаунта до работающего агента: настройка прав и секретов, подключение LLM к почте, собственный MCP-инструмент, авто-эскалация тикетов, RAG, безопасность и финальное ревью. Каждый шаг опирается на предыдущий. (Telegram — альтернативный канал, см. приложение к шагу 2.)

Упрощённо архитектура выглядит так (подробные схемы — в шагах про email-workflow и workflow):



Как устроена работа

- **Что делаем.** Собираем Help Desk-агента на Yandex AI Studio: агент на почте, RAG по базе знаний, тикеты и история диалогов в YDB (историю пишет poller и передаёт её агенту как контекст), авто-эскалация зависших обращений.
- **Что используем.** Стек Yandex Cloud: модель YandexGPT через Responses API, хранилище YDB Serverless, инструменты через MCP Hub, деплой через Cloud Functions и Yandex Workflows, секреты — в Lockbox.



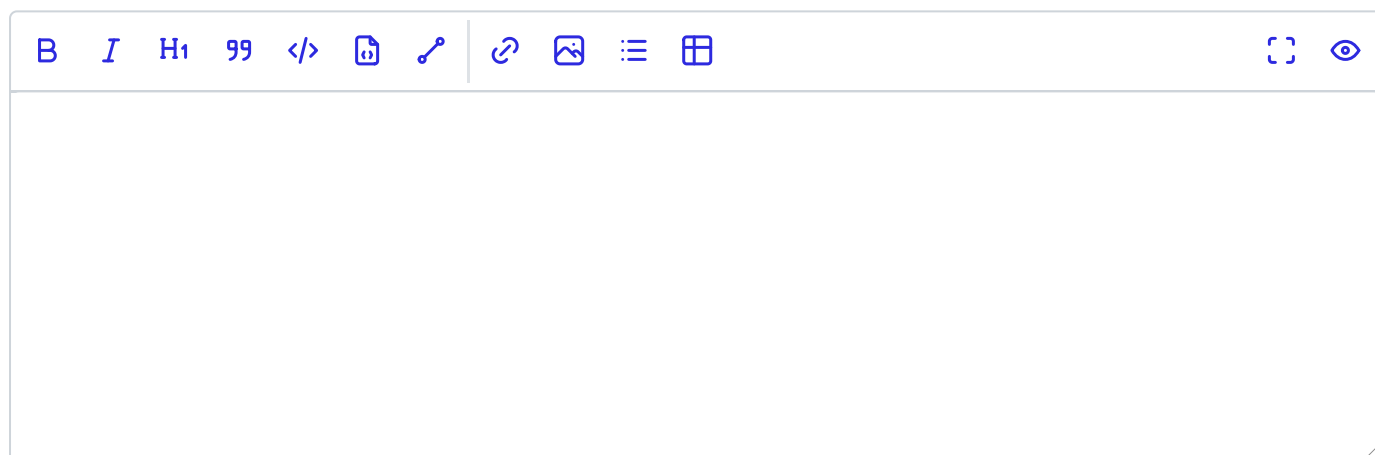
Все курсы

контракт сдачи от языка не зависит. Почтовая интеграция — в pull-режиме. СГ раз в минуту подключается к ящику по IMAP, забирает непрочитанные письма, отвечает по SMTP. Ответ приходит с задержкой до 60 секунд (один цикл таймера) — это ожидаемо. Проверяете сами: рабочий почтовый агент, скриншоты трейсов и YDB; формальные проверки раскладки файлов и секретов идут автоматически в CI. На сдачу — репозиторий с конфигами (*README.md*, *.env.example*, исходники в *src/*) и адрес ящика для проверяющего. Начнём со следующего шага — настройки окружения.

Результат шага

Вы разберётесь с задачей и предметной областью: поймёте, что за продукт предстоит собрать, из каких компонентов он состоит (агент + RAG + MCP/YDB + workflow) и как они связаны. Познакомитесь с моделью работы — pull-режим почтового poller'a и ручная проверка при сдаче — и подготовитесь к настройке облачного окружения.

ЗАВЕРШЕНО



Отправить

Комментарии

Комментариев пока нет

Хекслет

О нас

Направления

Программирование